



Chapter 4

ProD: A Tool for Predictive Design of Tailored Promoters in *Escherichia coli*

Friederike Mey, Jim Clauwaert, Maarten Van Brempt, Michiel Stock, Jo Maertens, Willem Waegeman, and Marjan De Mey

Abstract

A major goal in synthetic biology is the engineering of synthetic gene circuits with a predictable, controlled and designed outcome. This creates a need for building blocks that can modulate gene expression without interference with the native cell system. A tool allowing forward engineering of promoters with predictable transcription initiation frequency is still lacking. Promoter libraries specific for σ^{70} to ensure the orthogonality of gene expression were built in *Escherichia coli* and labeled using fluorescence-activated cell sorting to obtain high-throughput DNA sequencing data to train a convolutional neural network. We were able to confirm in vivo that the model is able to predict the promoter transcription initiation frequency (TIF) of new promoter sequences. Here, we provide an online tool for promoter design (ProD) in *E. coli*, which can be used to tailor output sequences of desired promoter TIF or predict the TIF of a custom sequence.

Key words Synthetic biology, Machine learning, Promoter design, Metabolic engineering, Biotechnology, Toolbox

1 Introduction

As metabolic engineering advances in both fundamental knowledge on the functioning of microorganisms as well as in providing tools to achieve precise engineering goals, the required level of precision increases simultaneously [1–3]. To improve current production titers by engineering a desired expression level, it is often not sufficient to find and turn the right screw. Combinatorial pathway optimization has been one strategy to reduce the time invested in searching for the optimal pathway which maximizes production [4, 5]. In general, control can be executed on three levels: transcriptional, translational and post-translational. While libraries exist

Authors Friederike Mey and Jim Clauwaert have equally contributed to this chapter.

These authors jointly supervised this work: Willem Waegeman, Marjan De Mey

Eveline Peeters and Indra Bervoets (eds.), *Prokaryotic Gene Regulation: Methods and Protocols*, Methods in Molecular Biology, vol. 2516, https://doi.org/10.1007/978-1-0716-2413-5_4, © The Author(s), under exclusive license to Springer Science+Business Media, LLC, part of Springer Nature 2022

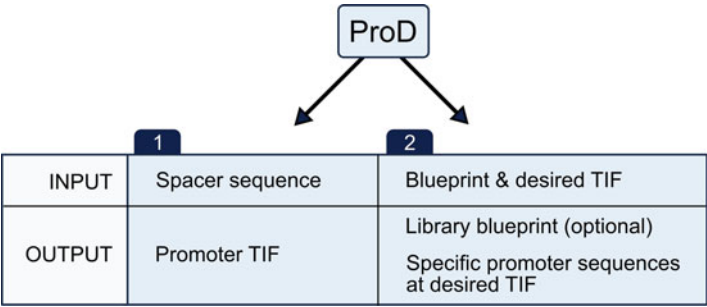


Fig. 1 The two-way usage of ProD, a promoter transcription initiation frequency (TIF) prediction tool supporting de novo design of promoter sequences. ProD can perform 2 types of predictions: In (1), the tool takes a custom spacer sequence and predicts the TIF. In (2), an input blueprint and desired TIF are used to output the blueprint for a library at the desired TIF, as well as specific spacer sequences for each requested TIF, or a single spacer sequence for each requested TIF

with promoters and ribosome binding site (RBS) sequences at predefined strengths [6], such libraries lack the flexibility that might be required in new setups. The success of the RBS (Library) Calculator from the Salis lab [7, 8] has shown the high demand of the synthetic biology community for such predictive tools that output a de novo library. In this work, we present ProD, a promoter transcription initiation frequency (TIF) prediction tool supporting de novo design of promoter sequences. This tool was built using promoters with randomized spacer regions in *Escherichia coli* that remained orthogonal by being specific to sigma factor σ^{70} . Orthogonality was ensured by leaving the -35 and -10 boxes adjacent to the fully randomized spacer region intact [9]. Data was collected by cloning the promoters upstream a fluorescent reporter gene, fluorescent-activated cell sorting (FACS) and subsequent DNA sequencing, allowing for the collection of considerably large datasets (250,000–400,000 unique sequences per setting) that mapped each sequence to fluorescence output. A custom-built convolutional neural network for ordinal regression was then trained on this dataset which is able to evaluate a custom spacer sequence, returning an estimated promoter TIF (0–10) based on the DNA sequence. Alternatively, ProD is able to output a promoter spacer sequence library at desired TIF or a library with specific promoter sequences in the range of the desired TIF (0–10, or very low to very high expression) [10]. An overview of the two workflows is depicted in Fig. 1.

2 Materials

ProD is available at <https://github.com/MEMO-group/ProD> and written in Python 3.7. The tool can either be run through Binder, which does not require a local installation, or installed on a local machine (*see Note 1*). To ensure compatibility in the latter case, the required package versions are listed in the repository.

2.1 Relevant Repository Files

1. ProD.py: the actual python script for the predictive promoter tool (*see Note 2*).
2. ProD_Notebook.ipynb: a Jupyter Notebook used for running code in an interactive environment. Contains detailed instructions on the functionality of the tool.
3. README.md: markdown readme file containing instructions on how to use the tool.
4. environment.yml: contains the package versions with which the tool was built. This custom environment ensures smooth running of the program. For ProD, Conda is used for environment management and therefore provided as a YAML file.
5. ex_seqs.fa: example input file used in the README.MD file. The fasta file is read as input and results in an output prediction csv file containing the predicted TIF for the input sequences (in this repository, it is called my_predictions.csv).

2.2 Prerequisites If ProD Is Run from the Terminal

1. Clone the ProD GitHub repository in your working directory (can also be downloaded as ZIP file from the website).
2. For Linux users, an environment can be created using the environment.yml file provided in the repository. For Mac and Windows users, verify that required Python libraries are available in your active environment:
 - (a) Pytorch 1.4.0
 - (b) Numpy 1.19.1
 - (c) Pandas 1.1.0
 - (d) Scipy 1.5.2

2.3 Prerequisites If ProD Is Utilized Using Jupyter

1. Install Jupyter Notebook or Jupyter Lab (<https://jupyter.org>).
2. Open ProD_Notebook.ipynb with Jupyter
3. A brief explanation on how to use the file is described in the file. In short, a code cell can be run by clicking on it and pressing Ctrl + Enter.

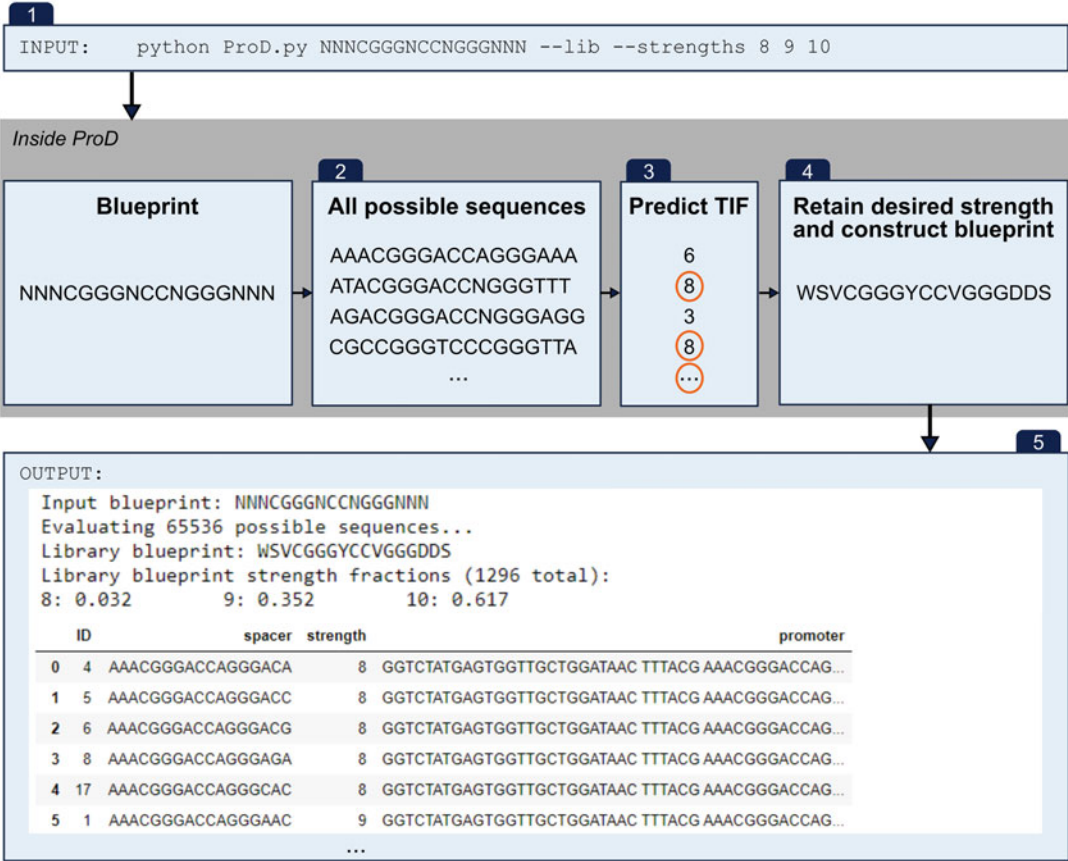


Fig. 2 The different steps in the process through which the ProD model obtains sequences with desired transcription initiation frequency (TIF). When constructing a library, the input (1) consists of a blueprint and desired TIF. The tool evaluates all possible sequences (2) and predicts their respective TIF (3). Only sequences of the desired TIF are retained from the previous step and a blueprint is constructed (4). The output (5) consists of the library blueprint, the fraction of sequences classified to each category of strength and a table with the spacer, TIF and insulator sequence. The CSV file `my_predictions.csv` further contains the probability for each TIF class for each spacer sequence

sequence or the library blueprint may be selected for subsequent cloning. The process through which the model obtains the final outputs is depicted in Fig. 2.

3.2.1 Using the Terminal

1. Open the terminal, activate the environment containing the packages described in Material.
2. Navigate to the directory containing the cloned GitHub repository.
3. Call the ProD tool inserting the desired library backbone and TIF(s), for example: `python ProD.py NNNCGGG NCCNGGGNNN --lib --strengths 8 9 10`. Alternative to a blueprint sequence, a fasta file containing a single blueprint

can be used as input: `python ProD.py my_blueprint.fa --lib --strengths 8 9 10`. If a single, specific spacer sequence is desired, add `--lib_size 1`.

4. The output is displayed in the terminal and additionally, a file `my_predictions.csv` is saved in the working directory. If another output path is desired, it can be specified in (3) by adding `--output_path PATH`

3.2.2 Using Jupyter

1. Open Jupyter in the environment containing the packages specified in Material and navigate to the directory containing the GitHub repository or load the ProD GitHub environment using the Binder cloud service.
2. Run the cell by selecting it and pressing `Ctrl + Enter`. The tool will be imported.
3. Enter your custom blueprint sequence in the next code cell as a string of 17 bases. Alternative to a blueprint sequence, a fasta file containing a single blueprint can be used as input: `input_data = ["my_blueprint.fa"]`.
4. Define the TIFs in `my_strengths` as comma-separated integers. If a single, specific spacer sequence is desired, add `lib_size=1` as argument of `run_tool`. Run the cell.
5. The output will be generated below the cell. This might take a while.
6. The generated csv file can be found in the working directory. If another output path is desired, it can be specified in **step 3** of this section by adding `output_path=PATH` as argument for `run_tool`.

3.2.3 Cloning Design

1. Select the library blueprint or a discrete spacer sequence.
2. Insert your selection in the insulator sequence (see Subheading 3.1 for reference of the insulator sequence).
3. Assemble and clone the resulting promoter sequence upstream of RBS + ORF of choice.

3.3 Predicting TIF by Evaluation of Custom Spacers

As discussed in Subheading 3.1, TIF can be dependent on the local context, which is why the prediction task is only expected to work reliably if the insulator sequence (except spacer) is kept intact.

3.3.1 Using the Terminal

1. Open the terminal, activate the environment containing the packages described in Material.
2. Navigate to the directory containing the cloned GitHub repository.
3. Evaluate one or multiple spacer sequences by calling for example `python ProD.py NNNCGGGNCCNGGGGAAA AAACCCGGGTTTAAAC`. Alternatively, the sequence (s) can be provided as fasta file: `python ProD.py ex_seqs.fa`

4. The output is displayed in the terminal and additionally, a file `my_predictions.csv` is saved in the working directory. If another output path is desired, it can be specified in the previous step by adding `-output_path PATH`.

3.3.2 Using Jupyter

1. Open Jupyter in the environment containing the packages specified in Material and navigate to the directory containing the GitHub repository or open Binder via the ProD GitHub website.
2. Locate the ‘Evaluate Custom Spacers’ section at the bottom of the notebook.
3. Provide input data either as list `input_data = ["AAAACCCCGGGGTTTTT", "TTTTCCCGGGGTTTTT", "TTTTGGGGCCCCAAAAA"]` or as file `input_file = ["ex_seqs.fa"]`
4. The output is displayed below the cell and additionally, a file `my_predictions.csv` is saved in the working directory. If another output path is desired, it can be specified in the previous step by adding `output_path=PATH` as argument for `run_tool`.

4 Notes

1. Binder is a service that can execute git repositories on the cloud for free. This can be achieved by visiting <https://mybinder.org>, or follow the link provided in the README. As such, no prerequisites are necessary. When using Binder, changes in the file are not saved.
2. The trained model is saved as `model_RPOD.pt` in the model folder and is used by the ProD tool to make predictions on novel input.
3. The 17 nucleotides sequence(s) are processed by a convolutional neural network of four 1×1 , $16 \times 1 \times 4$, and $32 \times 1 \times 2$ convolutions and two fully connected layers of 128 and 64 nodes.
4. When evaluating the model, no specific nucleotide was found to occur with high frequency that could not be explained by the input blueprint. Therefore, we cannot confirm the findings of other studies that a single nucleotide has an effect on promoter TIF [12–14], but rather suggest the influence of more complex structural features.
5. When creating a library, the tool first creates all possible sequences from the blueprint input, second, it predicts the TIFs of each sequence and retains those matching the requested TIF. Spacers are then sampled to construct a

degenerate sequence for the library blueprint (unless the input allows for more than 500,000 possible sequences). If the input sequence for generating libraries allows for more than 500,000 possible sequences, the tool will randomly sample 100,000 possible sequences to evaluate. The processing time might be longer, and no library blueprint will be generated.

6. While the original insulator was successfully tested on plasmid and genome level, the in vivo validation of our tool was performed on plasmids.
7. The input DNA sequence may contain any letter of the mixed-base code (R Y M K S W H B V D N).

Acknowledgements

F.M. holds a PhD grant (61478) from FWO (Fonds Wetenschappelijk Onderzoek – Vlaanderen). This research was also supported by the BOF-IOP project “MLSB” (BOF16/ IOP/040) of the Bijzonder Onderzoeksfonds, the FWO research project “SynSys-Bio4COS” (G0B8118) of the Research Foundation – Flanders (FWO) and by the H2020 project “BioRoboost” of the European Union (820699). W.W. also received funding from the Flemish Government under the “Onderzoeksprogramma Artificial Intelligence (AI) Vlaanderen” Programme.

References

1. Nielsen AA, Segall-Shapiro TH, Voigt CA (2013) Advances in genetic circuit design: novel biochemistries, deep part mining, and precision gene expression. *Curr Opin Chem Biol* 17:878–892. <https://doi.org/10.1016/j.cbpa.2013.10.003>
2. Bradley RW, Buck M, Wang B (2016) Tools and principles for microbial gene circuit engineering. *J Mol Biol* 428:862–888. <https://doi.org/10.1016/j.jmb.2015.10.004>
3. Brophy JAN, Voigt CA (2014) Principles of genetic circuit design. *Nat Methods* 11: 508–520. <https://doi.org/10.1038/nmeth.2926>
4. Jeschek M, Gerngross D, Panke S (2017) Combinatorial pathway optimization for streamlined metabolic engineering. *Curr Opin Biotechnol* 47:142–151. <https://doi.org/10.1016/j.copbio.2017.06.014>
5. Coussement P, Bauwens D, Maertens J, De Mey M (2017) Direct combinatorial pathway optimization. *ACS Synth Biol* 6:224–232. <https://doi.org/10.1021/acssynbio.6b00122>
6. De Mey M, Maertens J, Lequeux GJ et al (2007) Construction and model-based analysis of a promoter library for *E. coli*: an indispensable tool for metabolic engineering. *BMC Biotechnol* 7(34). <https://doi.org/10.1186/1472-6750-7-34>
7. Salis HM (2011) Chapter two – the ribosome binding site calculator. In: Voigt C (ed) *Methods in enzymology*. Academic Press, pp 19–42
8. Salis HM, Mirsky EA, Voigt CA (2009) Automated design of synthetic ribosome binding sites to control protein expression. *Nat Biotechnol* 27:946–950. <https://doi.org/10.1038/nbt.1568>
9. Bervoets I, Van Brempt M, Van Nerom K et al (2018) A sigma factor toolbox for orthogonal gene expression in *Escherichia coli*. *Nucleic Acids Res* 46:2133–2144. <https://doi.org/10.1093/nar/gky010>

10. Van Brempt M, Clauwaert J, Mey F et al (2020) Predictive design of sigma factor-specific promoters. *Nat Commun* 11:5822. <https://doi.org/10.1038/s41467-020-19446-w>
11. Davis JH, Rubin AJ, Sauer RT (2011) Design, construction and characterization of a set of insulated bacterial promoters. *Nucleic Acids Res* 39:1131–1141. <https://doi.org/10.1093/nar/gkq810>
12. Meng H, Wang J, Xiong Z et al (2013) Quantitative design of regulatory elements based on high-precision strength prediction using artificial neural network. *PLoS One* 8:e60288. <https://doi.org/10.1371/journal.pone.0060288>
13. Kiryu H, Oshima T, Asai K (2005) Extracting relations between promoter sequences and their strengths from microarray data. *Bioinformatics* 21:1062–1068. <https://doi.org/10.1093/bioinformatics/bti094>
14. Rhodius VA, Mutalik VK, Gross CA (2012) Predicting the strength of UP-elements and full-length *E. coli* σ^E promoters. *Nucleic Acids Res* 40:2907–2924. <https://doi.org/10.1093/nar/gkr1190>